The Hebrew University of Jerusalem

Department of Physics

N-body Gravitational Simulation

Barnes-Hut tree with adaptive RK4

Eran Rehani

ID 207823063

Course 77315: Computational Physics

2026

Abstract

I simulate 5000 equal-mass particles moving under their own gravity for 20 Gyr, at three starting speeds, $V = 65$, 80 and 95 km/s. The forces come from a Barnes-Hut octree with the assigned $\theta = 1$ test on the cell diagonal and the assigned softened force, and I step the particles forward with adaptive fourth-order Runge-Kutta, comparing one full step against two half steps.

The assigned accuracy, $\varepsilon = 0.1$ of the box size, allows a step error of 133.2 kpc. That is larger than the cluster's starting diameter of 100 kpc, so it barely limits the step size and I ran everything twice. Appendix A gives every required output at $\varepsilon = 0.1$, and the main text repeats the three runs with a tolerance 10^3 times tighter. I quote the energy error as $\max_t |\Delta E/E_0|$ of the escaper-corrected energy, which reaches 54% at the assigned tolerance and stays below 0.7% at the production tolerance, so the physical conclusions come from the production runs.

Contents

1 Problem and initial conditions

The system is $N = 5000$ equal-mass particles ($m = 2\times10^{7}\,M_\odot$, total mass $M = 1\times10^{11}\,M_\odot$) moving under their own gravity for 20 Gyr. At $t = 0$ they fill a sphere of radius $R = 50\,\mathrm{kpc}$ uniformly. To sample that sphere I draw three uniform random variables $u_r, u_\mu, u_\varphi \sim U(0,1)$ and map them to spherical coordinates:

$$r = R u_r^{1/3}, \qquad \cos\vartheta = 2u_\mu - 1, \qquad \varphi = 2\pi u_\varphi. \tag{1}$$

The cube root is what makes the sample uniform in volume rather than in radius. The volume element is $r^2\,dr$, so drawing r flat on $[0, R]$ would crowd particles into the center. With $u_r^{1/3}$ the number density comes out constant,

$$\rho = \frac{M}{\frac{4\pi}{3}R^3}. \tag{2}$$

Each Cartesian velocity component is Gaussian with standard deviation $\sigma = V/\sqrt{3}$, which sets the initial rms speed to $V = \sqrt{\langle v^2\rangle}$. The assignment asks for three cases, $V = 65$, 80 and 95 km/s. I draw the positions once and the velocities once, then reuse both draws in all three runs and rescale the velocities by V, so any difference between the runs comes from the starting kinetic energy and not from the random numbers. I subtract the mean velocity to remove the bulk motion of the cluster. I leave the positions alone, because shifting them can push particles outside the sphere the assignment specifies.

Particles interact via softened gravity:

$$\mathbf{f}_{ij} = -\frac{G m_i m_j}{(r_{ij} + S)^2}\frac{\mathbf{r}_{ij}}{r_{ij}}, \qquad \mathbf{r}_{ij} = \mathbf{r}_i - \mathbf{r}_j, \tag{3}$$

with softening length $S = 1\,\mathrm{kpc}$. Softening does two things here. It caps the pair acceleration at $|a_{ij}| \le G m_j/S^2$ instead of letting it diverge as $r_{ij} \to 0$, so one close approach cannot collapse the iteration on its own. It also stops particles from knocking each other off course when they pass too close. The 5000 simulation particles represent a smooth, collisionless mass distribution, so these close encounters are only a side effect of the simulation.

The potential energy that goes with this force is

$$U_{ij} = -\frac{G m_i m_j}{r_{ij} + S} \tag{4}$$

This potential matches the softened force in Equation 3, so the energy calculation is consistent with the simulation. After each accepted step I drop the particles that have left the cubic box $|x|, |y|, |z| \le 666\,\mathrm{kpc}$ ($L_{\mathrm{box}} = 1332\,\mathrm{kpc}$).

2 Numerical method

2.1 Units

I measure everything in kpc, $M_\odot$ and Gyr, which keeps the numerical values convenient. In these units,

$$G = 4.4996 \times 10^{-6}\,\mathrm{kpc}^3\,\mathrm{M}_\odot^{-1}\,\mathrm{Gyr}^{-2}, \qquad 1\,\mathrm{kpc/Gyr} = 0.977\,792\,2\,\mathrm{km/s}. \tag{5}$$

Each simulation particle has mass $m = M/N = 2 \times 10^{7}\,M_\odot$.

2.2 Barnes-Hut tree

Forces come from a Barnes-Hut octree [1]. The particles go into a cubic root cell. Any cell holding more than one particle splits into eight octants, and the splitting stops when a cell holds at most one. Every cell stores its total mass M_{cell}, its center of mass $\mathbf{r}_{\text{cm}}$, its particle count and its diagonal $L = \sqrt{3}\,l$. The tree pays off because I can replace a distant clump by one point mass at its center of mass, which brings a single force evaluation from $O(N^2)$ down to $O(N \log N)$.

For a given target particle, the tree search always opens a cell that contains the target, and it evaluates leaf cells directly. For a cell the target lies outside, at distance D from its center of mass, the algorithm recurses when $D/L < \theta$ and otherwise lumps the cell into a single point mass M_{cell} at $\mathbf{r}_{\text{cm}}$. The assignment defines the opening test using the cell diagonal $L = \sqrt{3}\,l$, while the usual definition uses the side length l. The two definitions are related by

$$\frac{D}{L} \geq \theta_{\text{diag}} \quad \Longleftrightarrow \quad \frac{l}{D} \leq \frac{1}{\sqrt{3}\,\theta_{\text{diag}}} \equiv \theta_{\text{side}}. \tag{6}$$

so the assigned $\theta_{\text{diag}} = 1$ is not the familiar $\theta = 1$. It is $\theta_{\text{side}} = 1/\sqrt{3} \approx 0.577$, which opens more cells than the usual choice, so it is both more accurate and slower. I rebuild the tree at every Runge-Kutta stage because the particle positions change between stages.

Dividing Equation 3 by the target mass gives the acceleration from one source particle or from a cell treated as a single mass:

$$\mathbf{a}_{ij} = -\frac{Gm_j}{(r_{ij} + S)^2} \frac{\mathbf{r}_{ij}}{r_{ij}}. \tag{7}$$

Figure 1 illustrates the method using a two-dimensional quadtree. It works like an octree but is easier to visualize. For 60 particles, the target requires 12 direct particle terms and 14 cell terms, giving 26 terms instead of 59.

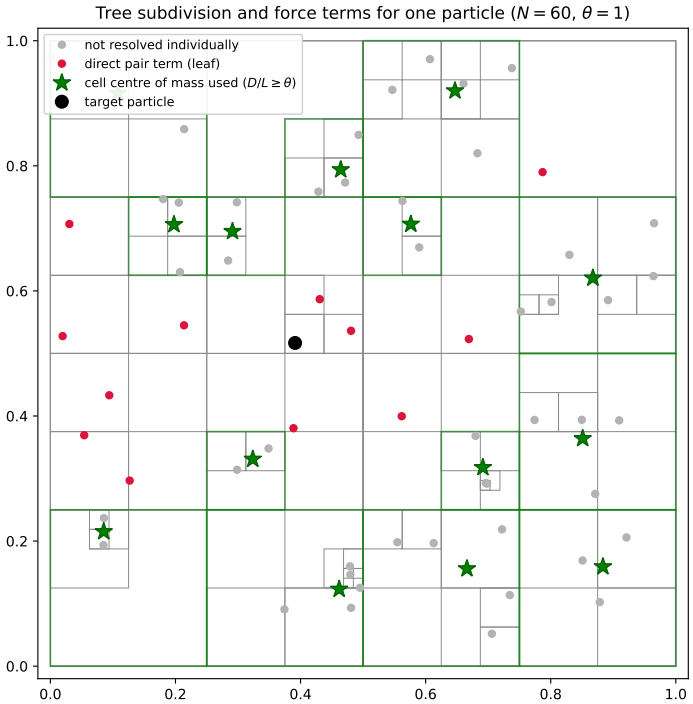

Figure 1: Two-dimensional illustration of the tree method with $\theta = 1$. Thin grey lines are cell boundaries. Red points contribute a direct pair force to the black target particle. Each green star is a cell center of mass that satisfies $D/L \geq \theta$, and the green square around it is the boundary of the cell it replaces. The star sits off-center inside it whenever the cell's particles do. Grey points are absorbed into those cells.

2.3 Adaptive RK4 integration

The state vector $\mathbf{y} = (\mathbf{r}_1, \ldots, \mathbf{r}_N, \mathbf{v}_1, \ldots, \mathbf{v}_N)^\top$ contains all particle positions and velocities. The equations of motion are $\dot{\mathbf{r}}_i = \mathbf{v}_i$ and $\dot{\mathbf{v}}_i = \mathbf{a}_i$. I control the RK4 step size using step doubling. For each proposed step h, I compare one full step with two half steps over the same time interval. Both calculations start from the same state and share the first stage $\mathbf{k}_1$. Reusing this stage reduces each proposed step from 12 force evaluations to 11 without changing the result.

The error estimate is the largest displacement difference over single particles:

$$\mathrm{err} = \max_{1 \le i \le N} \left\| \mathbf{r}_{i,\mathrm{half}} - \mathbf{r}_{i,\mathrm{full}} \right\|. \tag{8}$$

I measure the error for each particle rather than across the full $6N$ state vector. I do this because the assigned tolerance is a distance and should be compared with one particle's position error. When $\mathrm{err} \le \mathrm{tol}$ the step is accepted, and the two estimates are then combined by Richardson extrapolation:

$$\mathbf{y}_{\mathrm{acc}} = \mathbf{y}_{\mathrm{half}} + \frac{\mathbf{y}_{\mathrm{half}} - \mathbf{y}_{\mathrm{full}}}{2^4 - 1}, \tag{9}$$

This removes the main $O(h^5)$ error without any extra calculations. The next step size is then

$$h_{\mathrm{next}} = h \times 0.9 \left(\frac{\mathrm{tol}}{\mathrm{err}} \right)^{1/5}, \tag{10}$$

I limit each step-size change to a factor between 0.1 and 2, never shrink a rejected step below 0.01 Gyr, and never grow one past 2 Gyr. The lower limit prevents a close particle pair from making the step size so small that the simulation stalls. A step can still come out shorter than that when it is truncated to land exactly on an output time, and for the few steps that follow such a truncation while h grows back.

The assignment fixes the tolerance at $\varepsilon = 0.1$ of the box size:

$$\mathrm{tol} = \varepsilon L_{\mathrm{box}} = 0.1 \times 1332\,\mathrm{kpc} = 133.2\,\mathrm{kpc}. \tag{11}$$

This tolerance is larger than the cluster's initial diameter of 100 kpc. A step can therefore pass the test even when the particle positions are inaccurate. For $V = 80\,\mathrm{km/s}$ the energy error then reaches 54%, against 0.4% at the production tolerance. Section 4 explains that choice of 0.1332 kpc, and results using the assigned value are given in full in Appendix A.

2.4 Energy checks and escaper accounting

At each accepted step the code evaluates

$$E_k = \frac{1}{2} \sum_i m_i v_i^2, \qquad E_g = \frac{1}{2} \sum_i m_i \Phi_i, \qquad \Phi_i = -G \sum_{j \ne i} \frac{m_j}{r_{ij} + S}. \tag{12}$$

For $N > 1500$, I calculate the potentials Φ_i using the tree. For smaller systems, I use the exact direct sum because it is fast enough and avoids tree approximation errors.

When a particle leaves the box I drop it from the system, and its kinetic and potential energy go with it. The total energy then jumps, even though nothing physical has happened. To tell that jump apart from integration error, I record the particle's energy at the moment it leaves. The retained energy includes only the remaining particles, while the corrected energy adds the recorded energies of all escaped particles:

$$E_{\mathrm{corrected}}(t) = E_{\mathrm{retained}}(t) + \sum_{k \in \mathrm{escapers}} E_{\mathrm{esc},k}. \tag{13}$$

I remove a particle as soon as it crosses the box boundary, even if it is still bound to the cluster. For $V = 80$ km/s, 167 particles leave in 123 removal events. Of those, 21 have a total recorded energy at or below zero, $E_k + E_g \leq 0$.

2.5 Density profile and King fit

I measure each radius from the center of mass of the remaining particles and divide the particles into 25 logarithmically spaced shells. This gives smaller shells in the dense core and larger shells in the sparse outer region. For a shell with inner radius r_j and outer radius r_{j+1}, I divide the shell mass M_j by its volume to find the density. I plot this density at the geometric midpoint of the shell:

$$\rho_j = \frac{M_j}{\frac{4\pi}{3}(r_{j+1}^3 - r_j^3)}, \qquad M_j = N_j\, m, \qquad r_{j,\text{plot}} = \sqrt{r_j r_{j+1}}, \tag{14}$$

where N_j is the number of particles in the shell. Since the particles have equal masses, counting them gives the Poisson uncertainty $\delta\rho_j = \rho_j/\sqrt{N_j}$. The geometric midpoint fits here because both the shells and the radius axis are logarithmic.

I then fit the profile to the King model,

$$\rho(r) = \frac{\rho_c}{[1 + (r/r_c)^\alpha]^\beta}, \tag{15}$$

Here, ρ_c is the central density and r_c is the core radius. At large radii, the density falls as $\rho(r) \propto r^{-\alpha\beta}$, so the outer slope depends on the product $\alpha\beta$. I fit $\log\rho$ rather than ρ because the density spans several orders of magnitude. Fitting ρ itself would let the dense inner bins take over the fit. I weight each bin by $1/\delta\log\rho \approx \sqrt{N_j}$.

The assignment asks for the same α and β in all three cases, and allows two ways to do this. One way fits all four parameters to the $V = 80$ km/s run and uses its α and β for the other two. The other way fits the shared shape to all three runs at once, and that is the one I use. The three profiles go into a single least-squares problem with eight free parameters, a separate ρ_c and r_c for each velocity and one α and one β for all of them. The fit adds the χ^2 of the three profiles and minimizes the sum, so the three profiles pick the shape together. Taking the shape from $V = 80$ alone instead pushes the other two runs onto an outer slope that does not match their data. I ran both, since either one meets the requirement. Fitting the shape jointly lowers the total χ^2 from 1428 to 1166 over the same 65 degrees of freedom. Most of the gain is in the $V = 95$ km/s run, the worst case before, whose χ^2 drops from 984 to 508. Running `nbody_solver.py --refit` prints both.

3 Implementation and checks

I wrote the code in `nbody_solver.py`. To make it fast enough I store the tree in flattened arrays and let Numba compile the tree search, and I walk the tree for the target particles in parallel across threads, which is what makes the $N = 5000$ runs over 20 Gyr practical. The RK controller is the one I wrote for HW_05, changed to use the per-particle norm of Equation 8 and to resize the state vector when particles are removed. The isotropic direction sampler comes from HW_07.

Running `python nbody_solver.py --test` executes the checks described below, and `--tol=X` runs all three $N = 5000$ cases. I write the output to `figures/`, and there are also redraw and refit modes that rebuild the plots and the fitted tables from the saved states.

I first test the sampler using 5000 particles. The test sample has a maximum radius of 49.9905 kpc and an RMS speed of 79.422 km/s, while the production sample gives

49.9961 kpc and 80.235 km/s. Both maximum radii are below $R = 50$ kpc, and both speeds are within 1% of the target $V = 80$ km/s, as expected for a finite random sample. After subtracting the mean velocity, the center-of-mass velocity is zero to machine precision.

I check the tree against direct summation. For 150 particles with unequal masses at $\theta = 1$, the Barnes-Hut acceleration has a relative error $\|\mathbf{a}_{\text{tree}} - \mathbf{a}_{\text{dir}}\| / \|\mathbf{a}_{\text{dir}}\| = 1.014 \times 10^{-2}$, with both norms stacked over all particles and components. The tree gravitational energy differs from the direct result by 5.214×10^{-4}.

I test the integrator using problems with known solutions. I evolve a softened, unequal-mass circular binary for two orbits with a tight tolerance, and its relative energy change is only 1.29×10^{-10}, with a final separation that matches its initial value. A constant-velocity test also confirms that each snapshot is saved at the correct time.

For reference, the opening condition with $L = \sqrt{3}\, l$ is

$$\frac{D}{L} \geq \theta_{\text{diagonal}} \quad \Longleftrightarrow \quad \frac{l}{D} \leq \frac{1}{\sqrt{3}\, \theta_{\text{diagonal}}} \equiv \theta_{\text{side}}. \tag{16}$$

Table 1 compares the tree acceleration with a direct sum for 3000 particles from the initial sphere. At the assigned $\theta = 1$, the L_2 error is 9.14×10^{-3}, similar to the result of the smaller random test.

Table 1: Barnes-Hut acceleration error against direct summation for the initial uniform sphere, $N = 3000$.

θ (diagonal)	θ (side)	L_2 relative error in $\mathbf{a}$
2.0	0.289	1.17e-03
1.5	0.385	3.07e-03
1.0	0.577	9.14e-03
0.8	0.722	1.58e-02
0.6	0.962	3.55e-02
0.4	1.443	1.03e-01

Using $\theta = 2$ reduces the error by about a factor of eight with little extra runtime, because for $N = 5000$ most of the time goes into building the tree, so checking more cells adds little cost. The production runs use the assigned $\theta = 1$.

With the production tolerance, the controller accepted 310, 262 and 352 steps for $V = 80$, 95 and 65 km/s, and rejected 4, 10 and 3 proposed steps. The corresponding runs at the assigned tolerance rejected no steps, and the trial run of Table 2 rejects one, because it uses a smaller minimum step and no intermediate output times. This shows that the assigned tolerance is too loose to control the step size. The estimated error almost always remains below the large tolerance, so the controller accepts coarse steps and allows h to grow toward its maximum value. The simulation then uses too few steps to follow the particle motion accurately.

4 Choice of integration tolerance

The assigned tolerance, 133.2 kpc, is larger than the initial cluster radius, so I could not tell by inspection how much of the energy drift was physical and how much was integration error. To answer this, I repeated the $V = 80$ km/s simulation using four tolerances that span three orders of magnitude. For each run, the corrected energy is the retained energy plus the energy recorded for particles when they escape. This removes the jumps caused by deleting particles, but the force and energy still contain the same

Barnes-Hut approximation at every tolerance. Since only the tolerance changes between runs, the differences mainly show integration error.

Table 2: Energy drift versus RK4 position tolerance for $V = 80\,\mathrm{km/s}$, $N = 5000$, $20\,\mathrm{Gyr}$, including frozen escape values.

tol [kpc]	steps	rejected	N_f	max $\lvert\Delta E/E_0\rvert$
133.2	20	1	4781	4.82e-01
13.32	33	0	4833	8.37e-02
1.332	97	0	4829	1.77e-02
0.1332	303	4	4833	2.53e-03

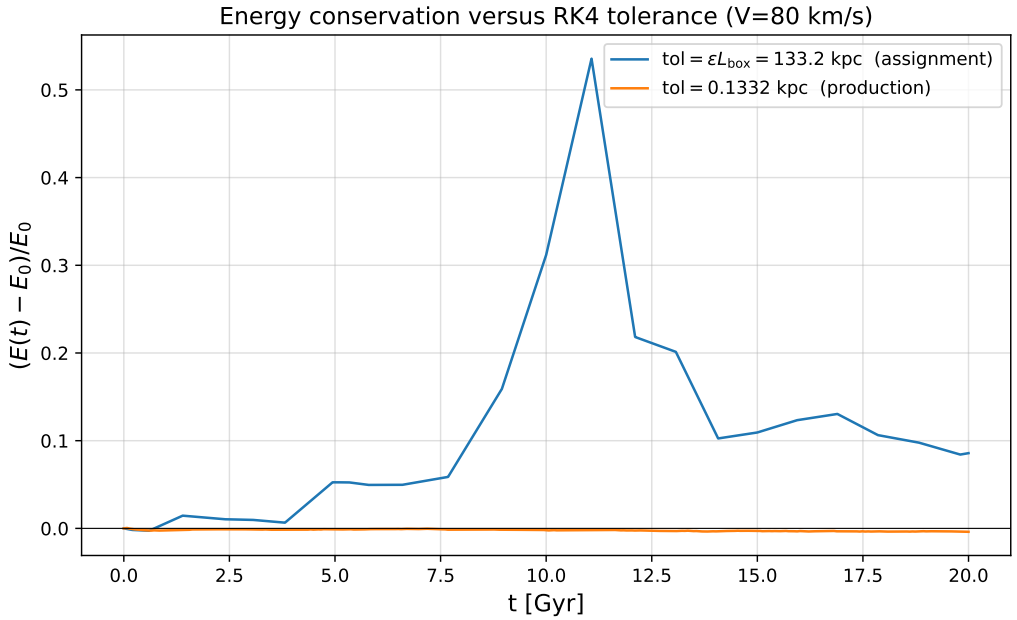

Figure 2: Relative energy change for $V = 80\,\mathrm{km/s}$ at the tolerance of the assignment and at the tolerance used for the production runs.

Within Table 2, tightening the tolerance by 10^3 lowers the worst energy error from 48% to 0.25% while the number of steps rises from 20 to 303. That is a factor of 190 in accuracy for a factor of 15 in work. These four trial runs use a smaller minimum step and no intermediate output times, so their numbers sit slightly below the production values quoted elsewhere (54% and 0.4% for the same two tolerances at $V = 80\,\mathrm{km/s}$). The trend is the same either way and that seemed worth it to me, so the production runs use $0.1332\,\mathrm{kpc}$. Appendix A includes every required output at the assigned tolerance under the suffix `_eps01`.

The drift is still falling at the last row of Table 2, so it has not yet reached a lower limit. With an even tighter tolerance, errors from the tree and close encounters would eventually take over, but these runs do not reach that point. I stopped because the remaining drift was already much smaller than the physical effects I was measuring.

5 Results

5.1 Initial energies and virial state

Before the simulations, I estimate whether each cluster starts above or below virial equilibrium. The virial theorem states that a self-gravitating system in equilibrium satisfies $2E_k + E_g = 0$, or $2E_k = |E_g|$. For an unsoftened uniform sphere, $E_g = -\frac{3}{5}\frac{GM^2}{R}$ and $E_k = \frac{1}{2}MV^2$. Therefore, the equilibrium speed is

$$V_{\text{vir}} = \sqrt{\frac{3GM}{5R}} = 71.85\,\text{km/s}. \tag{17}$$

The assigned speeds lie on both sides of this value, and that is why the three runs start out so differently.

- $V = 65$ km/s is sub-virial, $2E_{k,0}/\,|E_{g,0}| = 0.849$. There is not enough kinetic energy to hold the sphere up, so it collapses straight away.
- $V = 80$ km/s is super-virial, $2E_{k,0}/\,|E_{g,0}| = 1.286$. The excess kinetic energy pushes the outer layers out before gravity pulls the system back.
- $V = 95$ km/s is strongly super-virial, $2E_{k,0}/\,|E_{g,0}| = 1.814$. Its total energy is only just negative, so it is close to unbinding altogether and expands fast.

All three still start bound, $E_0 < 0$ (Table 3), so none of them simply disperses.

Table 3: Initial and final run data at the production tolerance tol = 0.1332 kpc. Energies use the simulation units.

V [km/s]	$2E_{k,0}/\,\lvert E_{g,0}\rvert$	E_0 [$10^{14} M_\odot\,\text{kpc}^2\,\text{Gyr}^{-2}$]	removed	retained	steps
65	0.849	−3.013	20	4980	352
80	1.286	−1.868	167	4833	310
95	1.814	−0.488	758	4242	262

5.2 (a) Particle positions

Figure 3 shows the particle positions at six times for $V = 80$ km/s. The sharp initial boundary quickly disappears. By $t = 2.5$ Gyr, the cluster has formed a dense core surrounded by a diffuse halo. The core then changes little, while the halo continues to expand. During the 20 Gyr simulation, 167 particles reach the box boundary and are removed. All six main panels share one ±175 kpc scale so that the core structure stays comparable between them, and the smaller view in the upper left of each panel repeats the same snapshot on the scale of the whole calculation box, with the particles outside the main panel drawn in red.

5.3 (b) Energy conservation

Figure 4 shows the relative energy change $\Delta E/E_0 = (E(t) - E_0)/E_0$ for $V = 80$ km/s. Since E_0 is negative, a positive value means that the total energy has become more negative. The retained energy (blue) and corrected energy (orange) agree until the first particle leaves the box at $t = 5.3$ Gyr.

During the 20 Gyr simulation, 167 particles escape. They remove kinetic energy equal to 4.6% of $|E_0|$, but only 0.8% in interaction energy because they are already far from

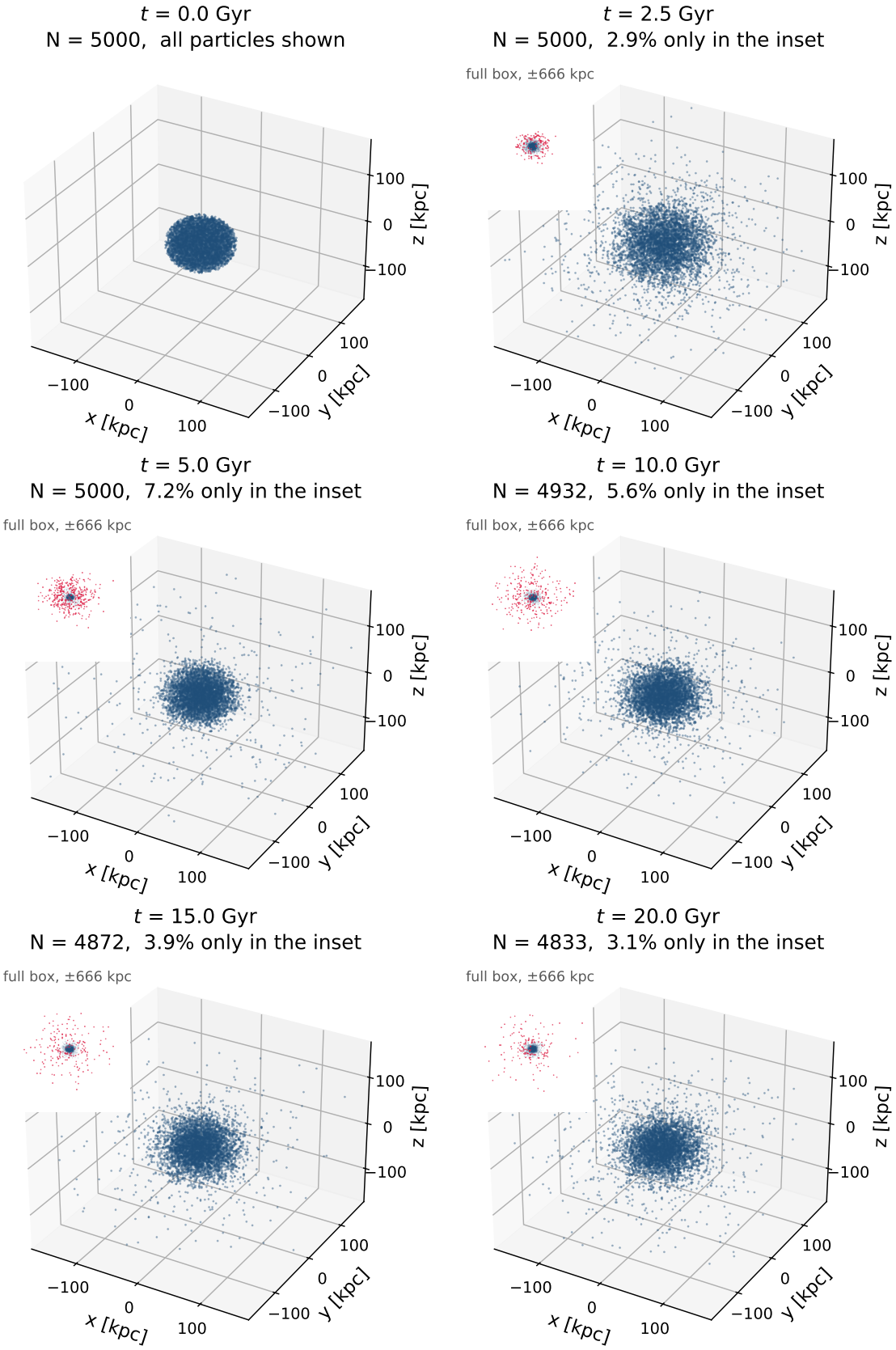

Figure 3: Particle positions for the $V = 80\,\mathrm{km/s}$ run. Each main panel shows the cluster core at the printed time, on a fixed $\pm 175\,\mathrm{kpc}$ scale. The small labeled cloud in the upper left of a panel is the same snapshot drawn on the scale of the whole calculation box, $\pm 666\,\mathrm{kpc}$, with the tail beyond the main panel in red. The percentage in the title is the fraction of particles that appear only there. The $t = 0$ panel already contains every particle and so carries no second view.

the cluster. Removing them therefore makes the remaining system appear more tightly bound, and the retained curve rises to +0.034. Adding their recorded energies back reduces the drift to −0.0039, or 0.39%. The effect is larger for $V = 95$ km/s, where 758 particles escape. In that run, the retained curve reaches +0.81, while the corrected curve ends at +0.0013.

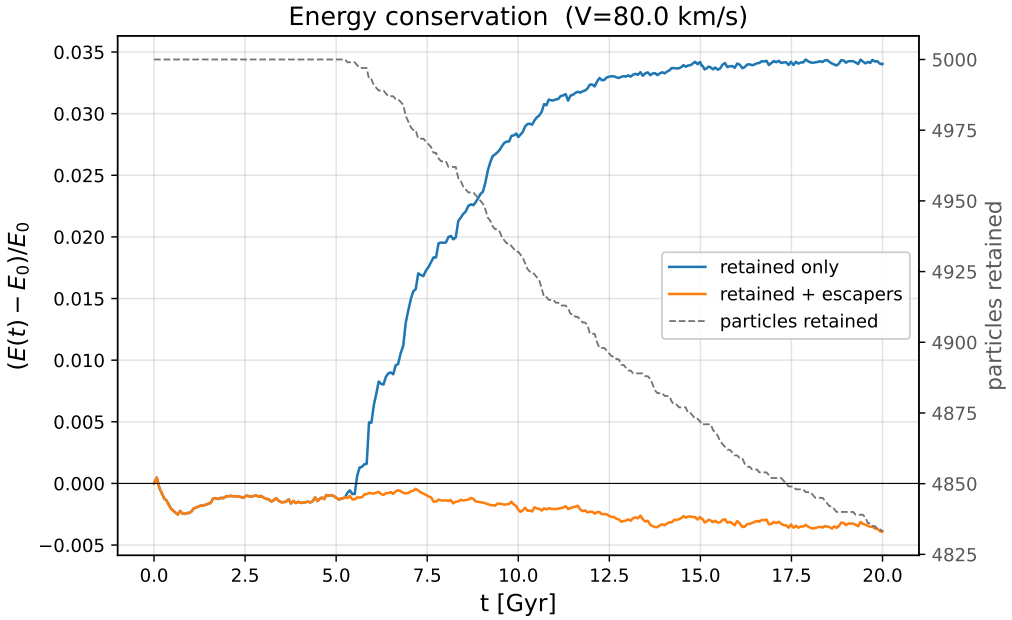

Figure 4: Relative energy change for $V = 80$ km/s. Blue uses retained particles, orange restores each escaper's energy recorded at removal, and the dashed grey line gives the retained count on the right axis.

5.4 (c) Virial ratio and violent relaxation

For $V = 80$ km/s, the ratio $-2E_k/E_g$ falls from 1.286 to 0.82 during the first 2.5 Gyr. It then oscillates around 1 with a decreasing amplitude (Figure 5), with peaks about 4.5 Gyr apart. This is a global breathing mode. The cluster expands past its equilibrium size, contracts again, and repeats with smaller oscillations.

The damping is a sign of violent relaxation [2]. When a collisionless system sits far from equilibrium, its gravitational potential $\Phi(\mathbf{r}, t)$ changes fast. Single particles then do not keep their own energy, and the changing potential moves them onto new orbits within one dynamical time, about $\tau_{\rm dyn} \sim 2.5$ Gyr here. That reshuffles the system far faster than close encounters between pairs of particles could. Once the potential settles, particle energies hold steady again, violent relaxation ends, and the oscillations fade.

At $t = 20$ Gyr, the retained ratios are 0.960, 0.954 and 0.937 for $V = 65$, 80 and 95 km/s. Values below 1 do not have to mean the clusters are out of equilibrium. The relation $-2E_k/E_g = 1$ is exact only for an unsoftened $1/r$ potential. Softening changes the potential to $U_{ij} \propto (r_{ij} + S)^{-1}$, so the virial term $\mathbf{r} \cdot \mathbf{f}$ is no longer equal to the potential energy U. The general equilibrium condition is therefore $2E_k + W = 0$, where W is the Clausius virial [3, 4]:

$$W = \sum_{i<j} \mathbf{r}_{ij} \cdot \mathbf{f}_{ij} = -\sum_{i<j} \frac{G m_i m_j r_{ij}}{(r_{ij} + S)^2}. \tag{18}$$

At the final time, $2E_k/|W|$ is 0.9996, 0.982 and 0.956, which `nbody_solver.py --virial` prints from the saved states. These values are closer to 1, confirming that the clusters are near virial equilibrium. The figure uses the ratio required by the assignment, while these corrected values account for softening.

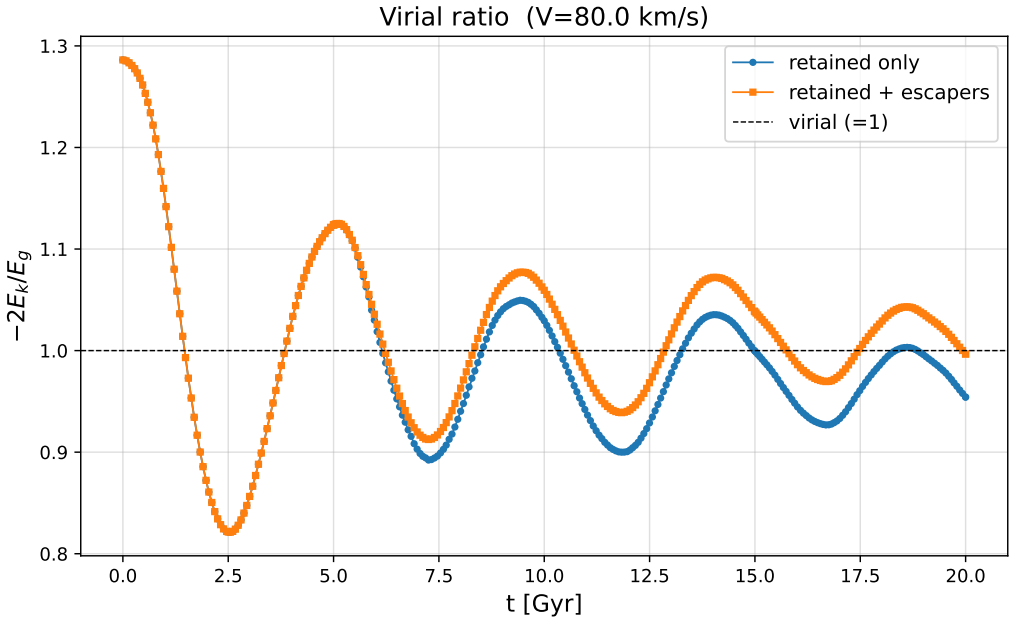

Figure 5: Virial ratio for $V = 80\,\mathrm{km/s}$. Blue uses the retained particles only, orange adds each escaper's energy recorded at removal, and the dashed line marks $-2E_k/E_g = 1$.

5.5 (d) Density profiles and King fits

Figure 6 shows the final radial density profiles and their King fits. As the initial speed increases, the core radius grows from 22.9 to 32.3 to 50.8 kpc, while the central density falls from 8.3×10^5 to $6.4 \times 10^4\ M_\odot\,\mathrm{kpc}^{-3}$. Faster particles travel farther from the center and spend more time at large radii, producing a wider and less dense core.

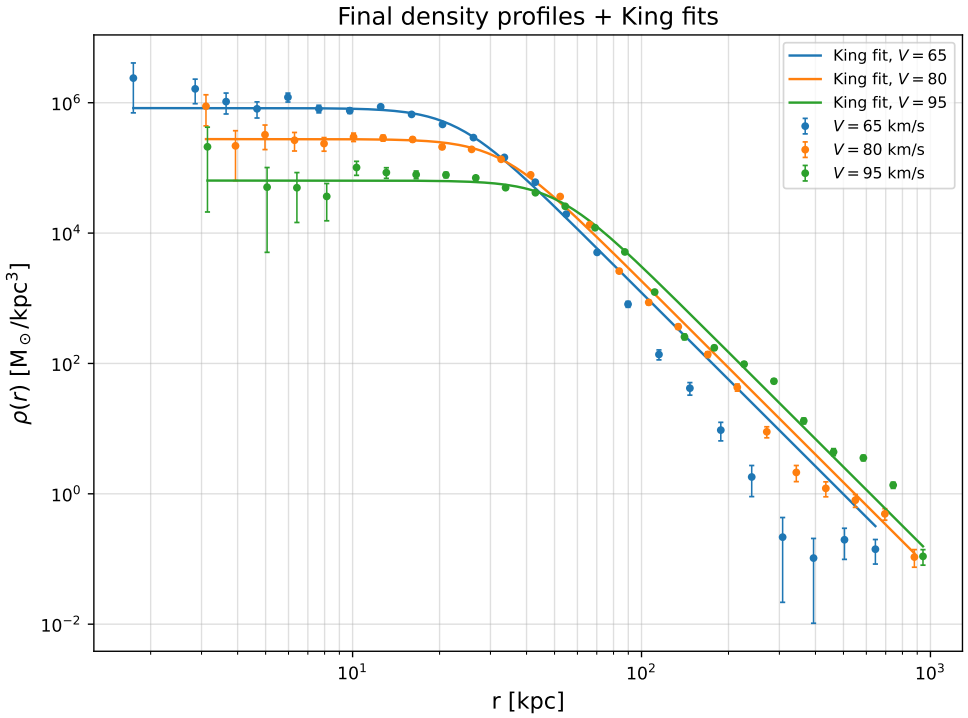

Figure 6: Center-of-mass density profiles at 20 Gyr with King-model fits. All three curves share one α and one β, fitted from the three profiles jointly. Only ρ_c and r_c differ between them.

Table 4: King-model parameters at the production tolerance tol = 0.1332 kpc. The uncertainties are one-standard-deviation errors from the weighted fit. α and β are shared by all three runs and are fitted from them jointly, so the same pair of values, and the same uncertainty on each, is listed in every row.

V [km/s]	ρ_c [$M_\odot$/kpc^3]	r_c [kpc]	α	β
80	2.766e+05 ± 1.10e+04	32.250 ± 0.377	4.501 ± 0.222	0.983 ± 0.053
95	6.396e+04 ± 2.51e+03	50.827 ± 0.652	4.501 ± 0.222	0.983 ± 0.053
65	8.274e+05 ± 3.16e+04	22.916 ± 0.249	4.501 ± 0.222	0.983 ± 0.053

The shared shape parameters are $\alpha = 4.50 \pm 0.22$ and $\beta = 0.983 \pm 0.053$. At large radii the density therefore falls as $\rho(r) \propto r^{-\alpha\beta} = r^{-4.43}$ for all three velocities, which is what makes the three fitted curves in Figure 6 parallel at large r. For comparison, fixing α and β at the values from a four-parameter fit to $V = 80$ alone gives $\alpha = 4.83 \pm 0.38$, $\beta = 0.986 \pm 0.085$ and an outer slope of 4.76. The two ways of fixing the shape agree on the outer slope to within the uncertainty on α.

The global χ^2 per degree of freedom is $1166/65 = 17.9$. The three runs contribute 256, 508 and 403 for $V = 80$, 95 and 65 km/s. These numbers are large because the error bars are small. The Poisson uncertainty falls as $\delta\rho \propto 1/\sqrt{N}$, so a shell holding thousands of particles has a tiny error bar, and even a small gap between the data and the King curve then counts for a lot. The fit also covers the whole radial range, past about 200 kpc, where the shells hold escaping particles rather than the settled cluster. The King model is not meant to describe those, which is why the outermost $V = 65$ km/s points in Figure 6 sit well below their fitted curve. The $V = 95$ km/s run still contributes the most after the joint fit, because of the dip in its central density that a single shared shape cannot follow.

6 Difficulties

6.1 Controlling long-term energy drift

The main difficulty I faced was keeping the energy stable during the full 20 Gyr simulation. With the assigned tolerance of 133.2 kpc, the RK4 controller accepted steps that were too large, and the energy error for $V = 80$ km/s reached 54%. I tested smaller tolerances and found that 0.1332 kpc reduced it to 0.4%, although it required many more steps. Because the assignment requires adaptive RK4, I solved the problem by using this tighter tolerance rather than changing the integration method.

6.2 Close encounters and duplicate coordinates

Close encounters produce large accelerations, which is why the controller needs a floor on the step size at all. Softening limits the acceleration to $|a_{ij}| \leq Gm_j/S^2$, and in the production runs that turned out to be enough. The floor only controls the shrinking of a rejected step, and no run ever needed it. Accepted steps ranged from 0.004 to 0.11 Gyr across the three velocities, with typical values near 0.07 Gyr. The shortest steps do not come from close encounters. They follow each output time, where the step is cut short to land on the breakpoint, and the controller can then grow it back by at most a factor of two per accepted step. A more serious problem occurs when two particles have exactly the same position, $r_{ij} = 0$. The octree cannot separate them, so a simple tree builder would continue splitting forever.

To prevent this, I stop splitting and store both particles in the same leaf. They act as one combined mass on other particles, and I set their mutual force to zero because its direction is undefined at $r_{ij} = 0$. The initial sampler is very unlikely to create duplicate positions, but this stops the recursion from running forever if duplicates do occur.

6.3 Potential evaluation and escaper energy

I use the tree for the potential, which costs $O(N \log N)$ instead of $O(N^2)$. At $N = 150$ it is off by 5.2×10^{-4} against a direct sum. This is much smaller than the integration drift, so the tree is not the main source of error.

My escaper correction is also only approximate. Recording each particle's energy when it leaves removes the false jumps in Figures 4 and 5, but ignores later changes in its potential energy. This missing contribution is small near the box boundary, but not exactly zero. Therefore, the corrected curve is a useful check on the energy rather than an exactly conserved quantity.

Related implementation. I also looked at the open-source `lechebs/nbody` project, which runs a Barnes-Hut simulation in real time using CUDA C++ and OpenGL. I used it only as a comparison with another implementation. All numerical results in this report come from my CPU/Numba code.

7 Conclusion

The Barnes Hut forces agree with direct summation to about 1% at $\theta = 1$, and the integrator passes the binary and constant-velocity tests. By 20 Gyr, all three clusters are close to virial equilibrium, with $2E_k/|W| \approx 0.96$-1.00. The sub-virial $V = 65$ km/s cluster contracts and retains 99.6% of its mass. The $V = 80$ and 95 km/s clusters expand and lose 3.3% and 15.2% of their mass.

The main change from the assignment is the tolerance. The assigned value of 133.2 kpc allows up to 54% energy error, while the production value of 0.1332 kpc keeps it below 0.7%. Appendix A includes the results at the assigned tolerance for comparison. In the production runs, a higher initial speed produces a larger core, $r_c = 22.9 \to 50.8$ kpc, and a lower central density.

A Results at the assigned tolerance

This appendix repeats the required outputs at the assigned tol $= \varepsilon L_{\text{box}} = 133.2$ kpc rather than the production value of 0.1332 kpc used in the body. These files carry the `_eps01` suffix and come from `python nbody_solver.py --tol=133.2 --tag=eps01`. The initial conditions, θ, S, box, and end time are unchanged.

At the assigned tolerance, the controller accepts 28, 15 and 34 steps for $V = 80$, 95 and 65 km/s and rejects none. These counts differ slightly from Table 2 because those runs use a smaller minimum step and no intermediate output times. Both sets of results show the same problem. The tolerance is larger than the initial cluster diameter, so it barely limits the step size.

Table 5: Comparison of the assigned and production tolerances. The error column is $\max_t |\Delta E/E_0|$ of the escaper-corrected energy, the convention used throughout this report. The corresponding values at $t = 20\,\mathrm{Gyr}$ alone are -0.347, $+0.086$ and $+0.219$ at the assigned tolerance and -0.003, -0.004 and $+0.001$ at the production one.

	tol = 133.2 kpc (assigned)			tol = 0.1332 kpc (production)		
V [km/s]	steps	retained	max $\lvert\Delta E/E_0\rvert$	steps	retained	max $\lvert\Delta E/E_0\rvert$
65	34	4755	0.469	352	4980	0.003
80	28	4822	0.536	310	4833	0.004
95	15	4239	0.223	262	4242	0.007

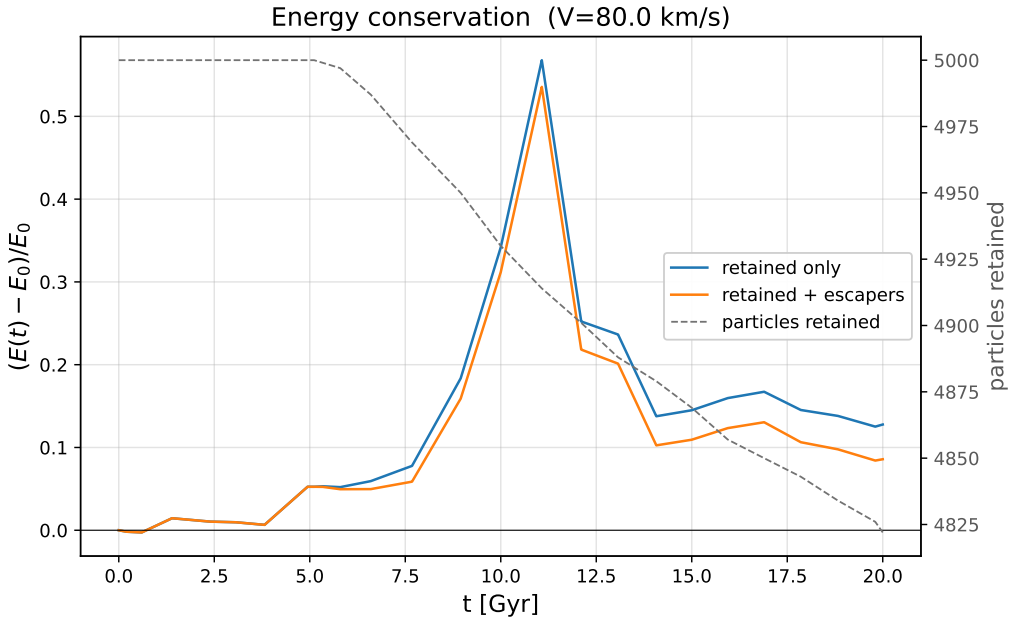

Figure 8: Energy and population for $V = 80\,\mathrm{km/s}$ using the assigned tolerance.

The corrected energy error reaches 54% in the full-output $V = 80$ run, against 48% for the trial run of Table 2 at the same tolerance. The difference comes from the smaller minimum step and the missing output breakpoints in that run. The loose run also removes 178 particles, compared with 167 in the production run.

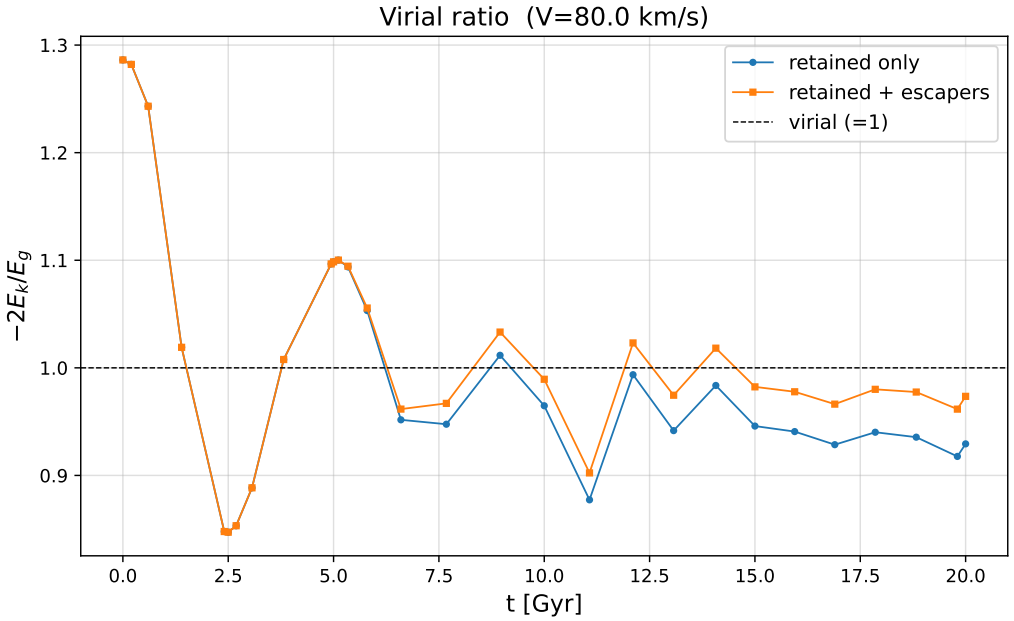

Figure 9: Virial ratio for $V = 80\,\mathrm{km/s}$ using the assigned tolerance, with the same two curves as Figure 5.

The virial ratio still ends near unity, at 0.929 instead of 0.954. With only 28 steps, the curve does not show the 4.5 Gyr oscillation, so the final value alone cannot show that the system reached equilibrium.

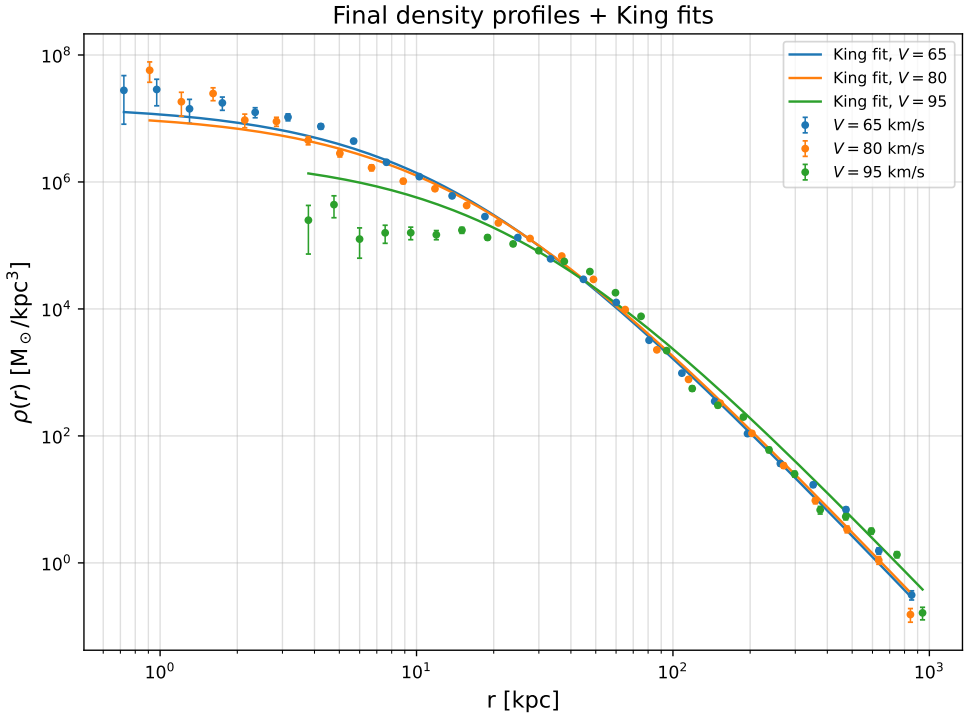

Figure 10: Density profiles and King fits using the assigned tolerance, with α and β again shared across the three velocities.

Table 6: King parameters using the assigned tolerance, fitted jointly with one shared α and one shared β, exactly as in Table 4.

V [km/s]	ρ_c [$M_\odot$/kpc^3]	r_c [kpc]	α	β
80	1.226e+07 ± 1.49e+06	14.561 ± 0.503	0.985 ± 0.041	4.334 ± 0.232
95	2.604e+06 ± 3.87e+05	24.179 ± 0.895	0.985 ± 0.041	4.334 ± 0.232
65	1.605e+07 ± 1.88e+06	13.228 ± 0.462	0.985 ± 0.041	4.334 ± 0.232

The loose tolerance also changes the density fits. Table 6 gives $r_c = 13.2$, 14.6 and 24.2 kpc for $V = 65$, 80 and 95 km/s, roughly half the values of the production runs, and the first two agree to within about one core radius of each other rather than being clearly separated. For $V = 65$ km/s, the loose run creates a central peak that is not real and raises the central density from 8.3×10^5 to $1.6 \times 10^7\, M_\odot\,\mathrm{kpc}^{-3}$, more than an order of magnitude. The shape parameters swap roles entirely, $(\alpha, \beta) = (0.985, 4.334)$ instead of $(4.501, 0.983)$, while their product, the outer slope, is nearly unchanged at 4.27 against 4.43. The outer slope is the one feature of the profile that survives the loose integration. The core structure does not.

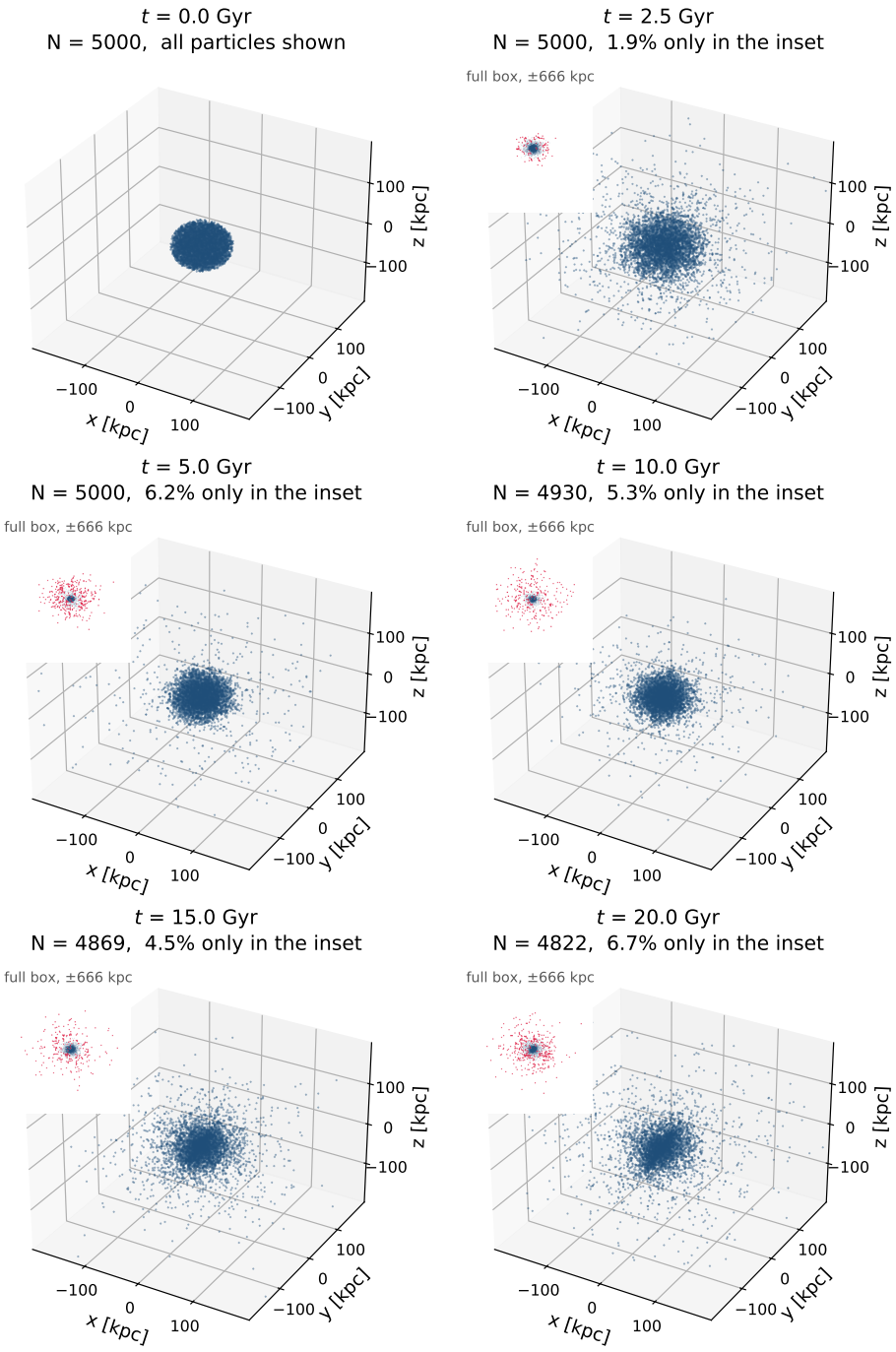

Figure 7: Particle positions for $V = 80\,\mathrm{km/s}$ using the assigned tolerance, laid out as in Figure 3. The main panels use a $\pm 200\,\mathrm{kpc}$ scale here. The labeled cloud in the upper left of each panel is the same snapshot on the full $\pm 666\,\mathrm{kpc}$ box scale, with the tail beyond the main panel in red.

References

[1] J. Barnes and P. Hut, "A hierarchical $O(N \log N)$ force-calculation algorithm," *Nature* **324**, 446-449 (1986).

[2] D. Lynden-Bell, "Statistical mechanics of violent relaxation in stellar systems," *Monthly Notices of the Royal Astronomical Society* **136**, 101-121 (1967).

[3] H. Goldstein, C. Poole and J. Safko, *Classical Mechanics*, 3rd ed. (Addison-Wesley, 2002), §3.4.

[4] J. Binney and S. Tremaine, *Galactic Dynamics*, 2nd ed. (Princeton University Press, 2008), Ch. 4.